

Modul 295 - Aufgabenblatt 06

von Lukas Meier

Unterricht vom 15.09.2026

1 Projekt vorbereiten

Erstellen Sie im `htdocs`-Ordner Ihrer XAMPP-Installation einen neuen Ordner `php-klassen` und speichern Sie die Datei `index.php`, welche Sie auf dem Netzwerkordner finden, darin. Öffnen Sie danach `http://localhost/php-klassen/` im Browser.

2 Erste Klasse

Definieren Sie eine Klasse `Product` mit den öffentlichen Eigenschaften `$name` (string) und `$price` (float) sowie einer Methode `printInfo()`, die mit `echo` Name und Preis im Format "**Apfel: 0.5 CHF**" ausgibt (verwenden Sie dazu `$this`). Erzeugen Sie zwei Objekte, setzen Sie deren Eigenschaften über den Pfeil-Operator `->` und rufen Sie bei beiden `printInfo()` auf.

3 Konstruktor

Ergänzen Sie die Klasse `Product` um einen Konstruktor `__construct($name, $price)`, der die beiden Eigenschaften direkt setzt. Erzeugen Sie die zwei Objekte aus Aufgabe 2 neu - dieses Mal, indem Sie Name und Preis direkt bei `new Product(...)` übergeben.

4 Constructor Property Promotion

Schreiben Sie den Konstruktor aus Aufgabe 3 mit **Constructor Property Promotion** um, sodass die Eigenschaften `$name` und `$price` direkt in der

Parameterliste des Konstruktors deklariert werden. Prüfen Sie, dass sich das Verhalten der Objekte nicht verändert hat.

5 Sichtbarkeit und Getter/Setter

Ändern Sie die Sichtbarkeit von `$price` auf `private`. Ergänzen Sie die Klasse `Product` um eine Methode `getPrice(): float`, die den Preis zurückgibt, sowie eine Methode `setPrice(float $price): void`, die den Preis nur übernimmt, wenn er mindestens 0 beträgt (sonst passiert nichts). Testen Sie `getPrice()`, rufen Sie `setPrice()` einmal mit einem gültigen und einmal mit einem negativen Wert auf und geben Sie den Preis danach jeweils erneut mit `getPrice()` aus.

6 Statische Eigenschaft und Methode

Ergänzen Sie `Product` um eine `private`, statische Eigenschaft `$count`, die im Konstruktor bei jeder Objekterzeugung um 1 erhöht wird, sowie eine öffentliche, statische Methode `getCount(): int`, die den aktuellen Wert zurückgibt. Erzeugen Sie insgesamt drei `Product`-Objekte und geben Sie danach `Product::getCount()` aus.

7 Klassenkonstante

Ergänzen Sie `Product` um eine öffentliche Konstante `TAX_RATE = 0.077` sowie eine Methode `priceWithTax(): float`, die den Preis inklusive Steuer zurückgibt (`self::TAX_RATE` verwenden). Geben Sie für eines Ihrer Produkte den Preis inklusive Steuer aus.

8 Vererbung

Erstellen Sie eine Klasse `SaleProduct`, die von `Product` erbt (`extends`). Sie soll zusätzlich eine öffentliche Eigenschaft `$discount` (`float`, als Rabatt zwischen 0 und 1) besitzen. Der Konstruktor von `SaleProduct` nimmt Name, Preis und Rabatt entgegen und ruft mit `parent::__construct(...)` den Konstruktor von `Product` auf, um Name und Preis zu setzen. Erzeugen Sie ein `SaleProduct`-Objekt und rufen Sie die geerbte Methode `printInfo()` darauf auf.

9 Methode überschreiben

Überschreiben Sie in `SaleProduct` die Methode `priceWithTax(): float` so, dass zuerst der Preis um den Rabatt reduziert wird (`price * (1 - discount)`) und erst danach die Steuer gemäss `TAX_RATE` aufgerechnet wird. Vergleichen Sie für denselben Ausgangspreis die Ausgabe von `priceWithTax()` bei einem normalen `Product` und einem `SaleProduct`.

10 Abstrakte Klasse

Definieren Sie eine abstrakte Klasse `Shape` mit einer abstrakten Methode `calculateArea(): float` sowie einer konkreten Methode `printArea(): void`, die mit `echo` die mit `calculateArea()` berechnete Fläche ausgibt. Erstellen Sie zwei Klassen `Circle` (mit Eigenschaft `$radius`) und `Rectangle` (mit Eigenschaften `$width` und `$height`), die von `Shape` erben und `calculateArea()` passend implementieren (`M_PI * radius ** 2` bzw. `width * height`). Erzeugen Sie je ein Objekt und rufen Sie `printArea()` auf.

11 Interface

Definieren Sie ein Interface `Describable` mit einer Methode `describe(): string`. Lassen Sie `Circle` und `Rectangle` dieses Interface zusätzlich zu `Shape` implementieren (`implements`) und ergänzen Sie beide Klassen um eine passende `describe()`-Methode (z.B. "Kreis mit Radius 3"). Legen Sie ein Array mit Ihren beiden `Shape`-Objekten an, durchlaufen Sie es mit `foreach` und geben Sie pro Objekt `describe()` sowie `printArea()` aus.

12 Readonly-Eigenschaften

Definieren Sie eine Klasse `Point` mit den öffentlichen, `readonly`-Eigenschaften `$x` und `$y` (float, über Constructor Property Promotion). Erzeugen Sie ein Objekt `new Point(3, 4)` und geben Sie `$x` und `$y` aus. Schreiben Sie als Kommentar dazu, was passieren würde, wenn Sie versuchen würden, `$p->x` nachträglich zu verändern.

13 instanceof

Legen Sie ein Array an, das Ihre `Circle`- und `Rectangle`-Objekte aus Aufgabe 10 sowie zusätzlich ein `Product`-Objekt enthält. Durchlaufen Sie das

Array mit `foreach` und geben Sie mit Hilfe von `instanceof` nur für die Objekte, die tatsächlich vom Typ `Shape` sind, `printArea()` aus.

14 Bonusaufgabe: Warenkorb mit Klassen

Erstellen Sie eine Klasse `ShoppingCart` mit einer privaten Eigenschaft `$items` (Array, initial leer). Ergänzen Sie eine Methode `addProduct(Product $product, int $quantity): void`, die ein Produkt zusammen mit der Menge im Warenkorb ablegt, eine Methode `getSubtotal(): float`, die über alle Positionen den Zwischenbetrag summiert (Preis mit `getPrice()` pro Produkt mal Menge), sowie eine Methode `getShipping(): float`, die 0 zurückgibt, wenn der Zwischenbetrag mindestens 15 beträgt, sonst 5.90. Ergänzen Sie ausserdem eine Methode `printReceipt(): void`, die pro Position eine Zeile im Format "Apfel: 6 x 0.5 = 3.00 CHF" ausgibt (Tipp: `number_format()`) und danach Zwischensumme, Versandkosten und den Gesamtbetrag inklusive Versand ausgibt. Erzeugen Sie mindestens drei `Product`-Objekte, legen Sie sie mit unterschiedlichen Mengen in einen `ShoppingCart` und rufen Sie `printReceipt()` auf.